Nantes Université

POLYTECH NANTES

Étude bibliographique : Attaques physiques par Side-Channel sur CPU RISC-V

24 octobre 2026



Enzo MACÉ & Lukas MOINET ETN5

Table des matières

1 Récupération d'un modèle de RAM **3**

2 Prise en main d'un modèle de processeur **10**

3 Création du système **16**

4 Question pour un champion en matériel **20**

Ajouter intro et conclusion

1

Récupération d'un modèle de RAM

1. Analyser le code récupéré ainsi que son testbench. Réaliser une simulation complète. Commentez les résultats.

Le composant `sync_ram` modélise une mémoire RAM à écriture et lecture synchrones sur front montant de l'horloge `clock`, fournie extérieurement.

- **Écriture :** Se fait lorsque `we = '1'`, la donnée présente sur le bus de données `datain` est écrite dans le tableau `ram` à l'indice pointé par le bus `address` au front montant de `clock`. L'adresse de lecture est également modifiée de la façon décrite ci-dessous.
- **Lecture :** L'adresse de lecture est enregistrée dans `read_address` à chaque front montant. La sortie `dataout` est assignée à `ram(to_integer(read_address))`, induisant un cycle d'horloge de latence en lecture.

Le fichier de test comporte deux architectures :

- **Bench** (Figure 1.1) : écrit en RAM la valeur `0x04` à l'adresse `0x001` à $t = 25$ns, puis `0x07` à l'adresse `0x002` à $t = 35$ns. En repassant `we = '0'` à $t = 35$ ns, la relecture de `0x001` (demandée à 45 ns) affiche `0x04` sur `DataOut` à 55 ns, et celle de `0x002` (demandée à 55ns) affiche `0x07` à 65ns.
- **Bench2** (Figure 1.2) : parcourt tout l'espace d'adressage pour écrire `DataIn = Address` avec `we = '1'`, puis relit séquentiellement chaque case mémoire. Le signal `ok` reste à `TRUE`, validant l'intégrité des écritures et lectures.

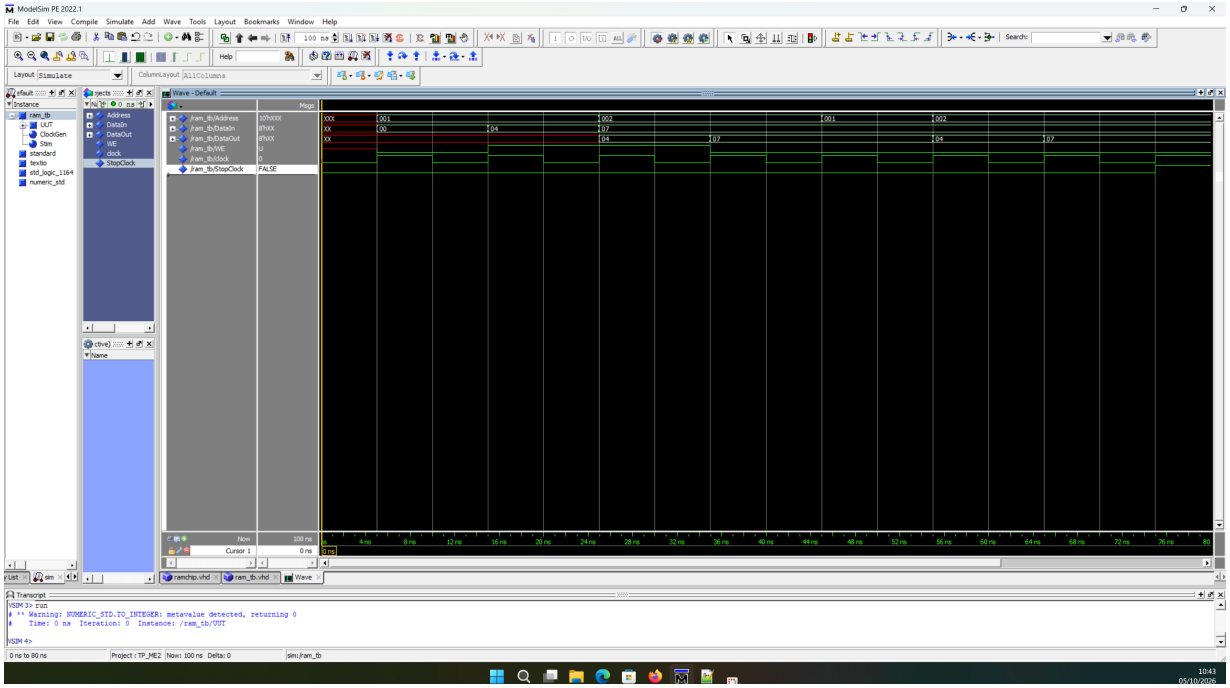


FIGURE 1.1 – Simulation initiale du bloc `sync_ram` avec l'architecture `Bench`.

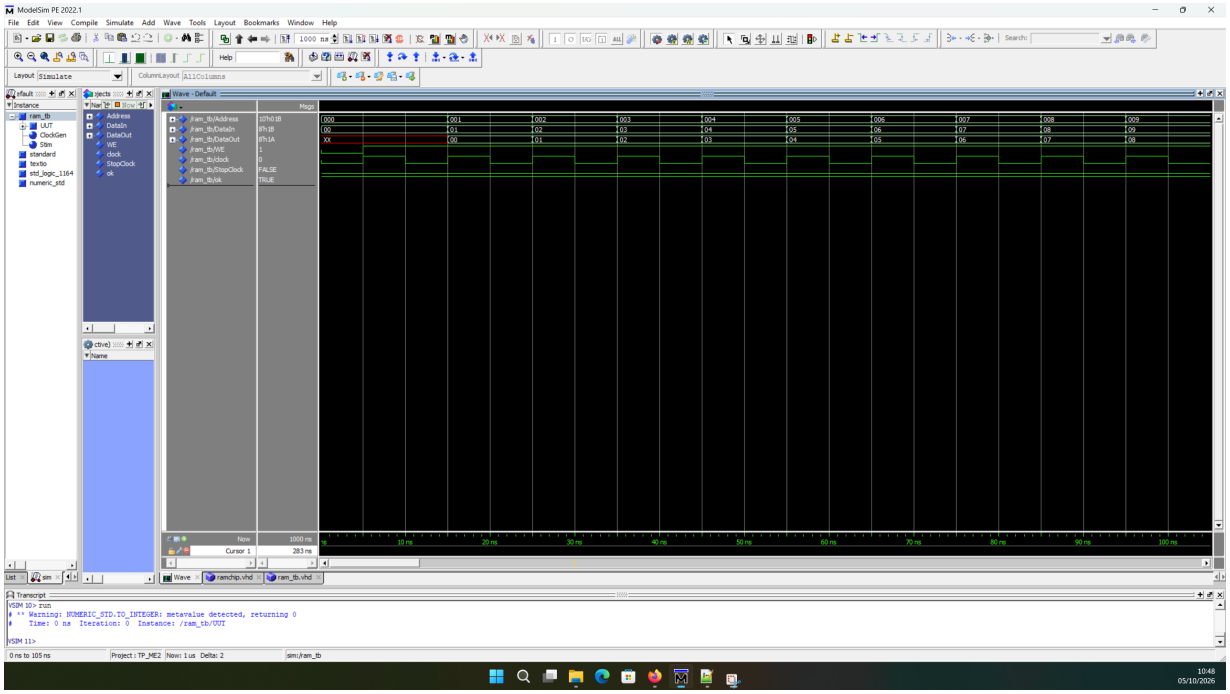


FIGURE 1.2 – Simulation initiale du bloc `sync_ram` avec l'architecture `Bench2`.

2. Utilisant le type `std_logic_vector()` ce modèle peut poser problème lors d'opérations arithmétiques et il convient donc de remplacer ce type par un type `unsigned()`.

Les ports `address`, `datain`, `dataout` ainsi que les signaux internes `ram` et `read_address` sont remplacés par le type `unsigned` (`IEEE.Numeric_Std.all`). Cela permet d'effectuer directement les incrémentations (`address + 1`) et conversions (`to_integer(address)`) sans cast intermédiaire.

3. Simulez le système pour vérifier que la modification n'a pas changé son fonctionnement.

Pour moi ça sert à rien de remettre ça, il faut juste dire que le comportement est exactement le même

Les chronogrammes obtenus après passage au type `unsigned` (Figures 1.3 et 1.4) sont identiques aux simulations initiales. Dans `Bench2`, `ok` se maintient à `TRUE`.

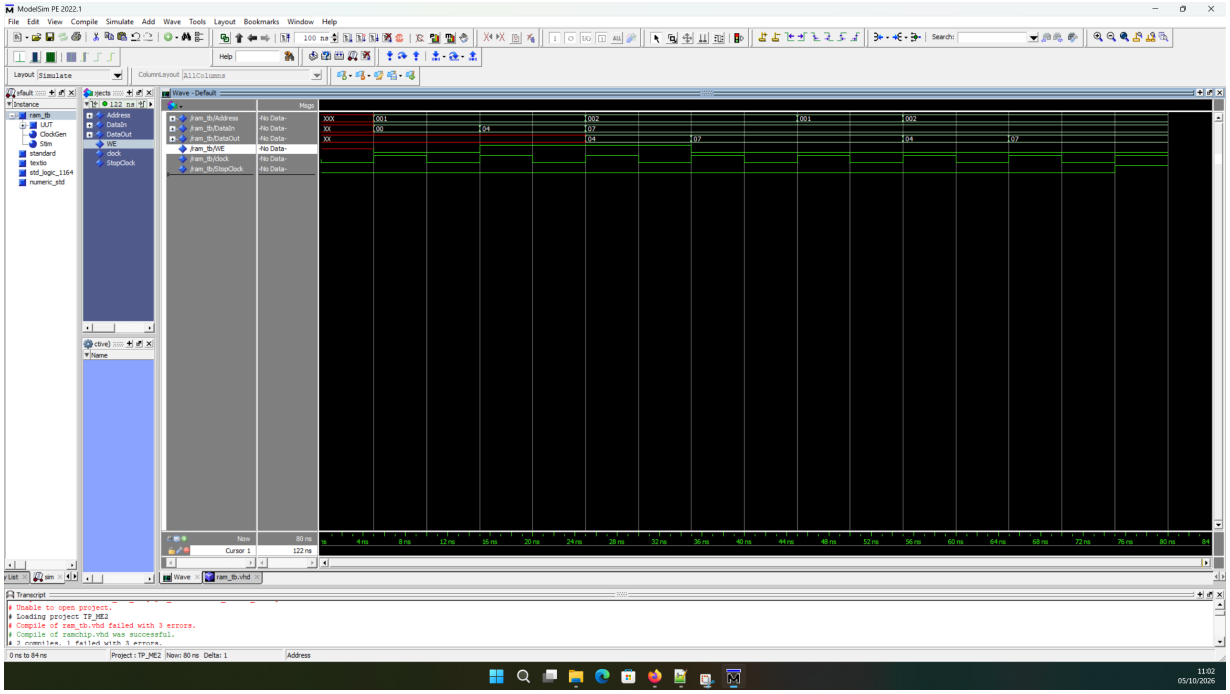


FIGURE 1.3 – Simulation de l'architecture `Bench` avec le type `unsigned`.

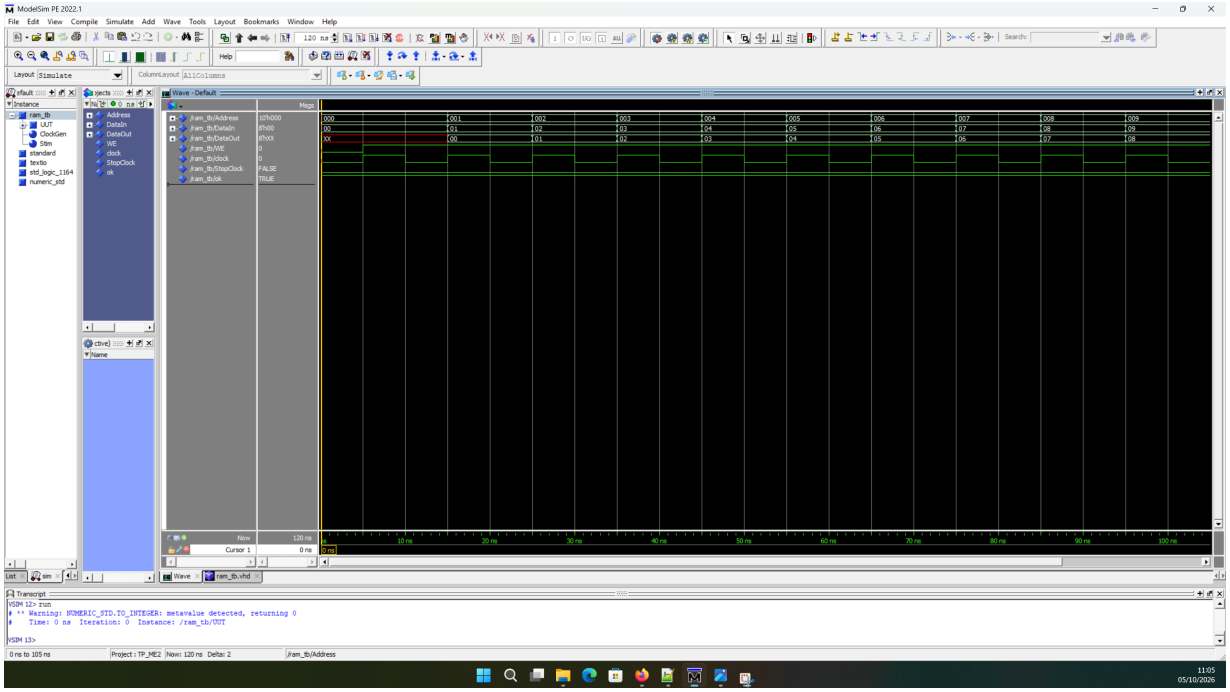


FIGURE 1.4 – Simulation de l'architecture `Bench2` avec le type `unsigned`.

4. On souhaite rendre ce bloc générique sur la largeur des données et sur le nombre de données mémorisables. Proposez une solution et implémentez-la.

Une clause `generic` est ajoutée à l'entité `sync_ram` avec les paramètres `NbBits` (largeur des mots) et `AddrLength` (largeur du bus d'adresse, fixant la capacité à $2^{\text{AddrLength}}$ mots).

La validation est effectuée via l'architecture `Bench16b` (`NbBits_cst = 16`, `AddrLength_cst = 20`). Sur la Figure 1.5, les valeurs 20 (`0x0014`) et 51 (`0x0033`) sont écrites aux adresses 1025 (`0x00401`) et 1026 (`0x00402`) à 25ns et 35ns, puis relues à 55ns et 65ns.

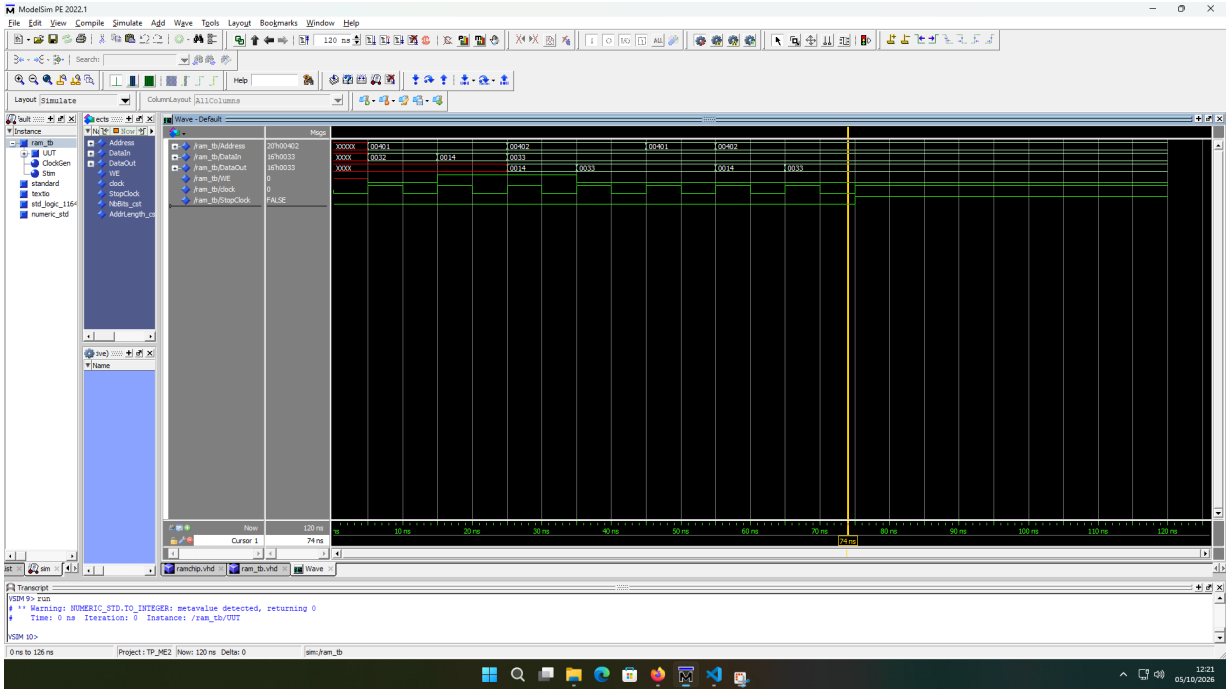


FIGURE 1.5 – Validation de la généricité (`Bench16b` : données sur 16 bits, adresses sur 20 bits).

5. Ajouter un signal CS asynchrone permettant de mettre les sorties de la mémoire à l'état haute impédance (état 'Z'). Réalisez les modifications et simulez la fonctionnalité de la mémoire incluant ce signal de sélection.

L'ajout du port `cs` conditionne l'affectation concurrente de `dataout`. Lorsque `cs = '1'`, la donnée pointée par `read_address` est placée sur le bus ; lorsque `cs = '0'`, la sortie passe de manière asynchrone à `(others => 'Z')`.

Listing 1.1 – Entité et architecture du bloc `sync_ram` générique avec `cs` (`ramchip.vhd`).

```
1  entity sync_ram is
2    generic (
3      NbBits      : integer := 8;
4      AddrLength  : integer := 8
5    );
6    port (
7      clock    : in std_logic;
8      we       : in std_logic;
9      address  : in unsigned(AddrLength - 1 downto 0);
10     datain   : in unsigned(NbBits - 1 downto 0);
11     dataout  : out unsigned(NbBits - 1 downto 0);
12     cs       : in std_logic
```

```
13    );
14  end entity sync_ram;
15
16  architecture RTL of sync_ram is
17    type ram_type is array (0 to (2 ** address'length) - 1) of
        unsigned(datain'range);
18    signal ram          : ram_type;
19    signal read_address : unsigned(address'range);
20  begin
21    RamProc : process (clock) is
22    begin
23      if rising_edge(clock) then
24        if we = '1' then
25          ram(to_integer(address)) <= datain;
26        end if;
27        read_address <= address;
28      end if;
29    end process RamProc;
30
31    dataout <= ram(to_integer(read_address)) when cs = '1' else
32               (others => 'Z');
33  end architecture RTL;
```

Sur le chronogramme de la Figure 1.6, le passage de `cs_ena` à `'0'` au cycle 7 ($t = 65$ ns) force immédiatement le bus `DataOut` à l'état haute impédance (trace bleue).

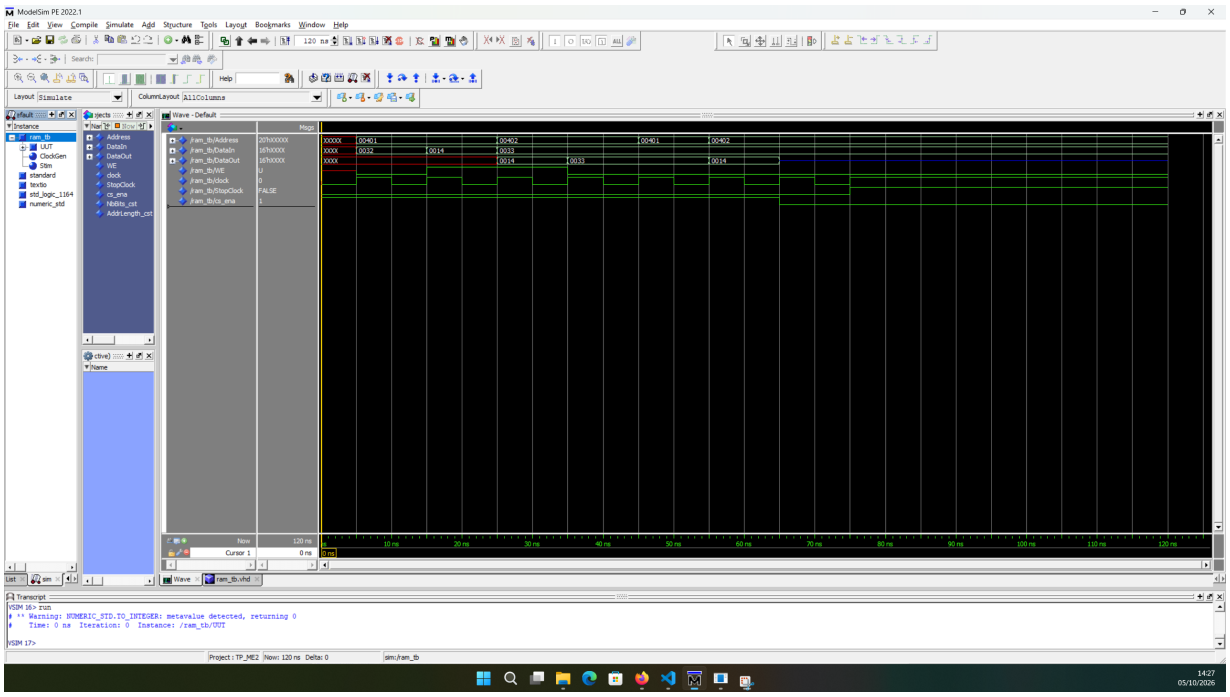


FIGURE 1.6 – Passage de `DataOut` à l'état haute impédance (`'Z'`) lorsque `cs_ena = '0'` à $t = 65$ ns.

2

Prise en main d'un modèle de processeur

2.0.1. Valider le processeur RISC-V.

Manipuler les instructions RISC-V. Modifier la description du processeur afin de charger ce programme dans la mémoire MEM et valider le fonctionnement du processeur.

Le chargement du programme machine (`instr.mem`) dans la mémoire interne `memory` s'effectue à l'initialisation via la tâche système Verilog `$readmemh`, suivi de la mise à zéro du bloc de registres `regs` :

Listing 2.1 – Initialisation de la mémoire et des registres dans le module `system`.

```
1 initial begin
2   pc       = 0;
3   addr_ram = 0;
4   $readmemh("instr.mem", memory);
5   for (index = 0; index < NRegs; index = index + 1) begin
6     regs[index] = 0;
7   end
8 end
```

Le programme de test initial (Figure 2.1) exécute la séquence suivante :

- `0x00200093` (`addi x1, x0, 2`) : $x1 = 2$.
- `0x00400113` (`addi x2, x0, 4`) : $x2 = 4$.
- `0x00110133` (`add x2, x2, x1`) : $x2 = 4 + 2 = 6$.

- `0x001150b3` (`srl x1, x2, x1`) : $x1 = 6 \gg 2 = 1$.
- `0x0020a523` (`sw x2, 10(x1)`) : écriture de $x2 = 6$ à l'adresse $x1 + 10 = 11$ (`0x00b`).
- `0x00a0a103` (`lw x2, 10(x1)`) : lecture à l'adresse 11 dans $x2$.
- `0x00100073` (`ebreak`) : arrêt de la simulation (`$finish`).

Sur le chronogramme de la Figure 2.2, les instructions sont chargées aux indices `memory[0]` à `memory[6]`. À $t = 50$ ns (`pc = 0x005`, instruction `sw`), `addr` prend la valeur `0x00b` (11) et la case `memory[11]` passe à `0x00000006`.

```
1   // instructions at address 0x0
2   // converter https://luplab.gitlab.io/rvcodecjs/#q=sw+x2,+10(x1)&abi=false&isa=AUTO
3   @0
4   00200093 // addi x1, x0, 2
5   00400113 // addi x2, x0, 4
6   00110133 // add x2, x2, x1
7   001150b3 // srl x1, x2, x1
8   0020a523 // sw x2, 10(x1)
9   00a0a103 // lw x2, 10(x1)
10  00100073 // ebreak
11
```

FIGURE 2.1 – Programme de test initial dans `instr.mem`.

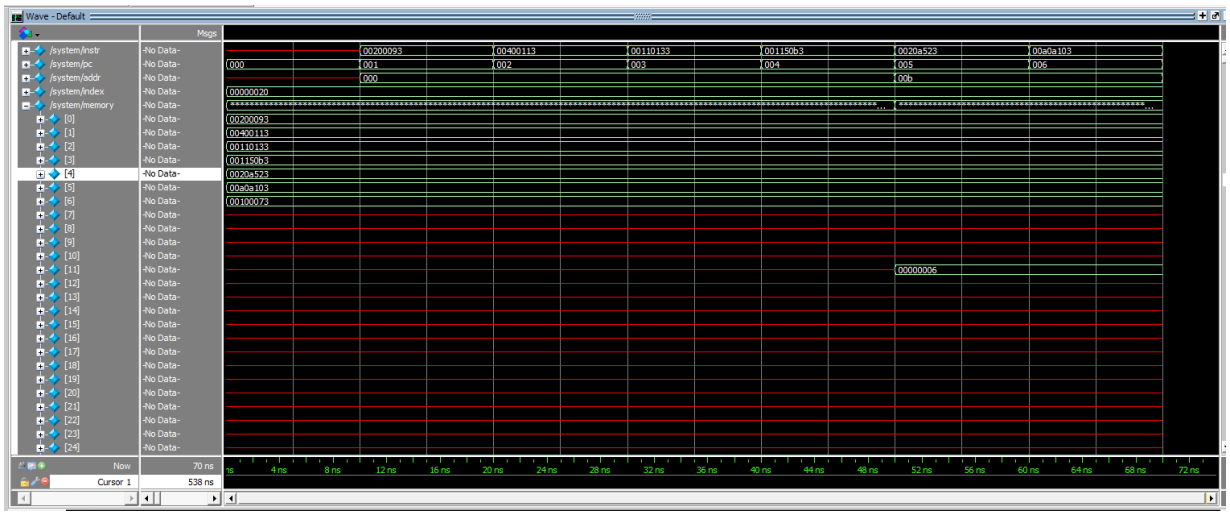


FIGURE 2.2 – Validation du processeur seul : exécution de `sw` et écriture de `0x00000006` dans `memory[11]` à $t = 50$ ns.

Adapter et implémenter les instructions RISC-V. 1. Expliciter les modes d'adressage direct, indirect, et immédiat.

- **Adressage immédiat :** La constante opérande est encodée directement dans les champs de l'instruction (`I_IMM` sur les bits `[31:20]` pour le type I). Exemple : `addi x1, x0, 2`.

- **Adressage direct (registre) :** Les opérandes sont lus et écrits dans les registres `regs` pointés par les champs `RS1` (`instr[19:15]`), `RS2` (`instr[24:20]`) et `RD` (`instr[11:7]`). Exemple : `add x2, x2, x1`.
- **Adressage indirect (base + déplacement) :** L'adresse mémoire effective est la somme de la valeur contenue dans `regs[RS1]` et de l'immédiat signé (`I_IMM` pour `lw`, `S_IMM` pour `sw`). Dans le modèle initial du processeur, un décalage de 12 bits (`LOAD_OFFSET` et `STORE_OFFSET`) était présent sur l'offset. Il a été commenté afin d'adresser directement `I_IMM + regs[RS1]` et `S_IMM + regs[RS1]`.

2. Le processeur fourni supporte le jeu d'instructions RV32I, expliquer quelles sont les modifications à réaliser permettant de supporter le jeu d'instructions RV32IM.

L'extension **RV32M** ajoute les instructions matérielles de multiplication (`mul`, `mulh`, `mulsu`, `mulu`) et de division/modulo (`div`, `divu`, `rem`, `remu`).

- Elles partagent l'opcode `ARITHMETIC` (`7'b0110011`) des instructions de type R, mais se distinguent par un champ `F7` égal à `7'b0000001`.
- Le couple `{F3, F7}` identifie chaque opération.
- D'après la spécification RISC-V (Table 7.1), les cas particuliers de division par zéro et de dépassement de capacité (*overflow* signé lors de la division de -2^{31} par -1) ne lèvent pas d'exception matérielle mais retournent des valeurs déterministes :
  - Division par 0 : `div` $= -1$, `divu` $= 2^{32}-1$, `rem` $=$ `rs1`, `remu` $=$ `rs1`.
  - Overflow signé ($-2^{L-1}/-1$) : `div` $= -2^{L-1}$ (`-2147483648`), `rem` $= 0$.

3. Implémenter ces modifications et modifier le programme de test pour valider vos modifications.

On déclare les macros `` `MUL `` à `` `REMU `` et on complète le `case({`F3, `F7})` du bloc `` `ARITHMETIC `` en traitant explicitement les cas de division par zéro et d'overflow :

Listing 2.2 – Déclarations et implémentation de l'extension RV32M.

```
1 `define MUL    {3'b000, 7'b0000001}
2 `define MULH   {3'b001, 7'b0000001}
3 `define MULSU  {3'b010, 7'b0000001}
4 `define MULU   {3'b011, 7'b0000001}
5 `define DIV    {3'b100, 7'b0000001}
6 `define DIVU   {3'b101, 7'b0000001}
7 `define REM    {3'b110, 7'b0000001}
8 `define REMU   {3'b111, 7'b0000001}
9
```

```
10 // Dans case({`F3, `F7}) :
11 `MUL:    regs[`RD] = (regs[`RS1] * regs[`RS2]);
12 `MULH:   regs[`RD] = {{32'd0, regs[`RS1]} * regs[`RS2]} >> 32;
13 `MULSU:  regs[`RD] = {{32'd0, regs[`RS1]} * regs[`RS2]} >> 32;
14 `MULU:   regs[`RD] = {{32'd0, regs[`RS1]} * regs[`RS2]} >> 32;
15 `DIV:
16   if (regs[`RS1] == -2**(DataWidth-1) && regs[`RS2] == -1)
17     regs[`RD] = -2**(DataWidth-1);
18   else
19     regs[`RD] = (regs[`RS2] != 32'd0) ? regs[`RS1] / regs[`RS2] : -1;
20 `DIVU:   regs[`RD] = (regs[`RS2] != 32'd0) ? regs[`RS1] / regs[`RS2] :
     2**32 - 1;
21 `REM:
22   if (regs[`RS1] == -2**(DataWidth-1) && regs[`RS2] == -1)
23     regs[`RD] = 32'd0;
24   else
25     regs[`RD] = (regs[`RS2] != 32'd0) ? regs[`RS1] % regs[`RS2] :
         regs[`RS1];
26 `REMU:   regs[`RD] = (regs[`RS2] != 32'd0) ? regs[`RS1] % regs[`RS2] :
     regs[`RS1];
```

Le programme `instr.mem` est enrichi (Figure 2.3) pour tester les cas limites sur 32 bits ($L = 32$) :

1. **Division et reste par** 0 ($x1 = 1, x3 = 0$) :
   - `0x0230e033` (`rem x0, x1, x3`) : $x0$ prend la valeur du dividende $x1 = 1$ à $t = 15$ ns.
   - `0x0230c033` (`div x0, x1, x3`) : $x0$ prend la valeur -1 à $t = 17$ ns.
2. **Division et reste de** -2^{31} **par** -1 :
   - `0x01910113` (`addi x2, x2, 25`) : $x2 = 6 + 25 = 31$ à $t = 19$ ns.
   - `0x002090b3` (`sll x1, x1, x2`) : $x1 = 1 \ll 31 = -2147483648$ (-2^{31}) à $t = 21$ ns.
   - `0x0200e133` (`rem x2, x1, x0`) : reste de $-2^{31}/-1$, $x2$ passe à 0 à $t = 23$ ns.
   - `0x0200c133` (`div x2, x1, x0`) : quotient de $-2^{31}/-1$, $x2$ passe à -2147483648 à $t = 25$ ns.

Ces résultats sont confirmés par le chronogramme de la Figure 2.4.

instr.mem

```
1   // instructions at address 0x0
2   // converter https://luplab.gitlab.io/rvcodecjs/#q=sw+x2,+10(x1)&abi=f
3   @0
4   00200093 // addi x1, x0, 2
5   00400113 // addi x2, x0, 4
6   00110133 // add x2, x2, x1
7   001150b3 // srl x1, x2, x1 // x1 = 1
8   0020a523 // sw x2, 10(x1)
9   00a0a103 // lw x2, 10(x1)
10
11  // division et reminder par 0
12  0230e033 // rem x0, x1, x3  // x0 = x1 = 1 (rem par 0)
13  0230c033 // div x0, x1, x3  // x0 = -1 (div par 0)
14
15  // division et reminder de -2**(L-1) par -1
16  01910113 // addi x2, x2, 25 // x2 = 31
17  002090b3 // sll x1, x1, x2 // x1 = -2**(L-1)
18  0200e133 // rem x2, x1, x0 // x2 = 0 (rem overflow)
19  0200c133 // div x2, x1, x0 // x2 = -2**(L-1) (div overflow)
20
21  00100073 // ebreak
```

FIGURE 2.3 – Programme de test des instructions `div` et `rem` (division par 0 et overflow).

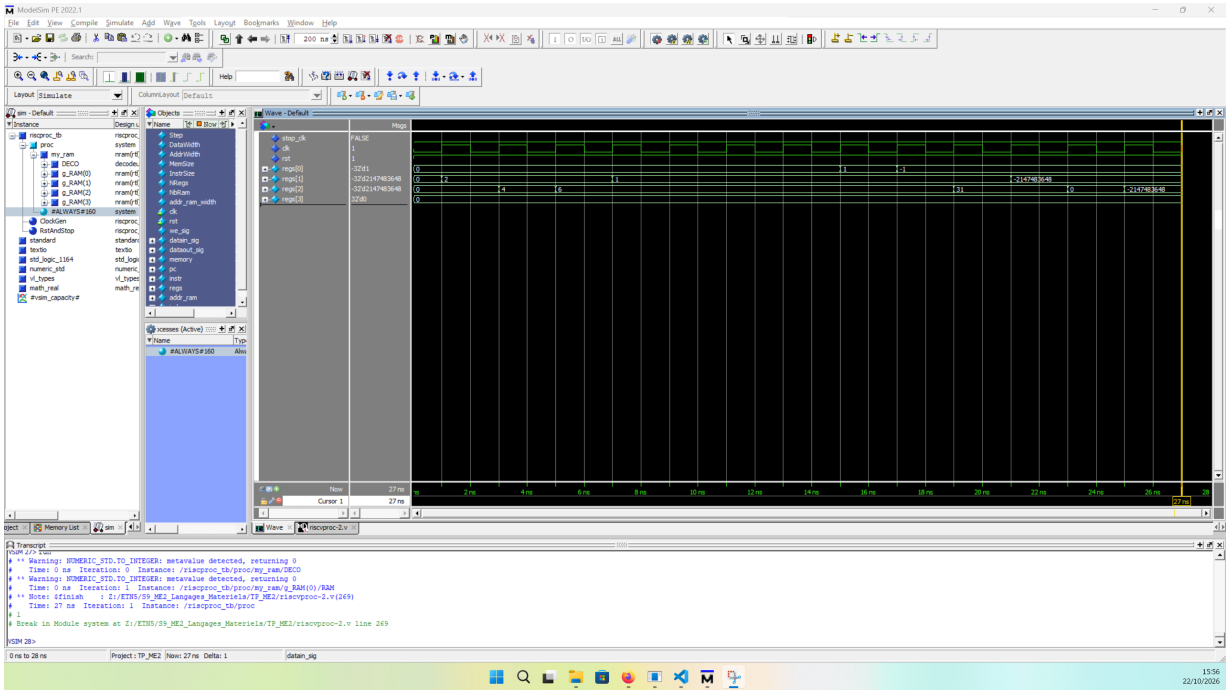


FIGURE 2.4 – Chronogramme de validation des instructions `div` et `rem` (RV32M).

2.0.2. Adapter la μ-architecture du processeur RISC-V.

Modifier la description de l'architecture matérielle du processeur afin de pouvoir l'interfacer avec le bloc quadram.

Le module Verilog est renommé `system(clk, rst)` et intègre les signaux d'horloge `clk` et d'initialisation `rst` en entrée, pilotés par le testbench VHDL `riscproc_TB.vhd`. La boucle d'exécution `always` est cadencée sur le front montant de l'horloge : `always @(posedge clk)`.

Pour interfacer le bloc mémoire `nram` composé de $\texttt{NbRam} = 4$ blocs de taille $2^{\texttt{AddrWidth}}$ mots ($\texttt{AddrWidth} = 12$), la largeur totale du bus d'adresse de la RAM est définie par :

$$\texttt{addr_ram_width} = \texttt{AddrWidth} + \log_2(\texttt{NbRam}) = 12 + 2 = 14 \text{ bits}$$

Les signaux internes d'interconnexion suivants sont déclarés dans le module `system` :

- `we_sig` (`reg`, 1 bit) : signal de contrôle d'écriture connecté au port `we`.
- `addr_ram` (`reg`, 14 bits) : bus d'adresse connecté au port `address`.
- `datain_sig` (`reg`, 32 bits) : bus de données d'écriture connecté au port `datain`.
- `dataout_sig` (`wire`, 32 bits) : bus de données de lecture connecté au port `dataout`.

3

Création du système

1. Modification des instructions de manipulation des données, 2. Gestion du temps de synchronisation requis par `nram`, et 3. Instanciation et interfaçage dans `system`.

Le composant VHDL `nram` est instancié directement à l'intérieur du module Verilog , lui-même stimulé par le testbench VHDL `riscproc_TB.vhd` (horloge `clk` de période 2 ns).

Listing 3.1 – Instanciation directe de l'entité VHDL `nram` et gestion des accès `LW/SW` dans `system`.

```
nram #(
  .NbRam(NbRam),
  .NbBits(DataWidth),
  .AddrLength1Cell(AddrWidth)
) my_ram(
  .clock(clk),
  .we(we_sig),
  .address(addr_ram),
  .datain(datain_sig),
  .dataout(dataout_sig)
);

// Modification des instructions LOAD et STORE :
`LOAD: begin
  case(`F3)
    `LW: begin
      addr_ram  = `I_IMM + regs[`RS1];
      regs[`RD] = dataout_sig;
```

```
19      end
20    endcase
21  end
22  `STORE: begin
23    case(`F3)
24      `SW: begin
25        we_sig     = 1'b1;
26        addr_ram   = `S_IMM + regs[`RS1];
27        datain_sig = regs[`RS2];
28        @ (posedge clk)
29        we_sig     = 1'b0;
30      end
31    endcase
32  end
```

- **Séparation des mémoires :** La mémoire interne `memory` ne contient plus que les instructions chargées par `$readmemh`, tandis que les données sont lues et écrites dans l'instance `my_ram` (`nram`).

- **Synchronisation de l'écriture (`SW`) :** Comme le processeur et la mémoire `sync_ram` sont déclenchés sur le même front montant de `clk`, l'instruction `sw` positionne `we_sig = 1'b1`, l'adresse `addr_ram` et la donnée `datain_sig`, puis attend le front montant suivant via `@(posedge clk)` avant de repasser `we_sig = 1'b0`. Ce cycle d'attente laisse le temps à la `sync_ram` d'écrire la donnée en mémoire et de mettre à jour sa sortie `dataout_sig` avant l'exécution de l'instruction suivante.

Sur le chronogramme de la Figure 3.1, lors de l'exécution de `sw x2, 10(x1)` (`instr = 0x0020a523`) à $t = 45$ ns, `we_sig` passe à `'1'`, `addr_ram` vaut `0x000b` (11, sélectionnant le bloc mémoire `g_RAM(0)` avec `cs = '1'`) et `datain_sig` vaut `0x00000006`. Au front montant suivant ($t = 53$ ns, curseur jaune), la case `g_RAM(0)/RAM/ram(11)` et la sortie `dataout_sig` prennent la valeur `0x00000006`.

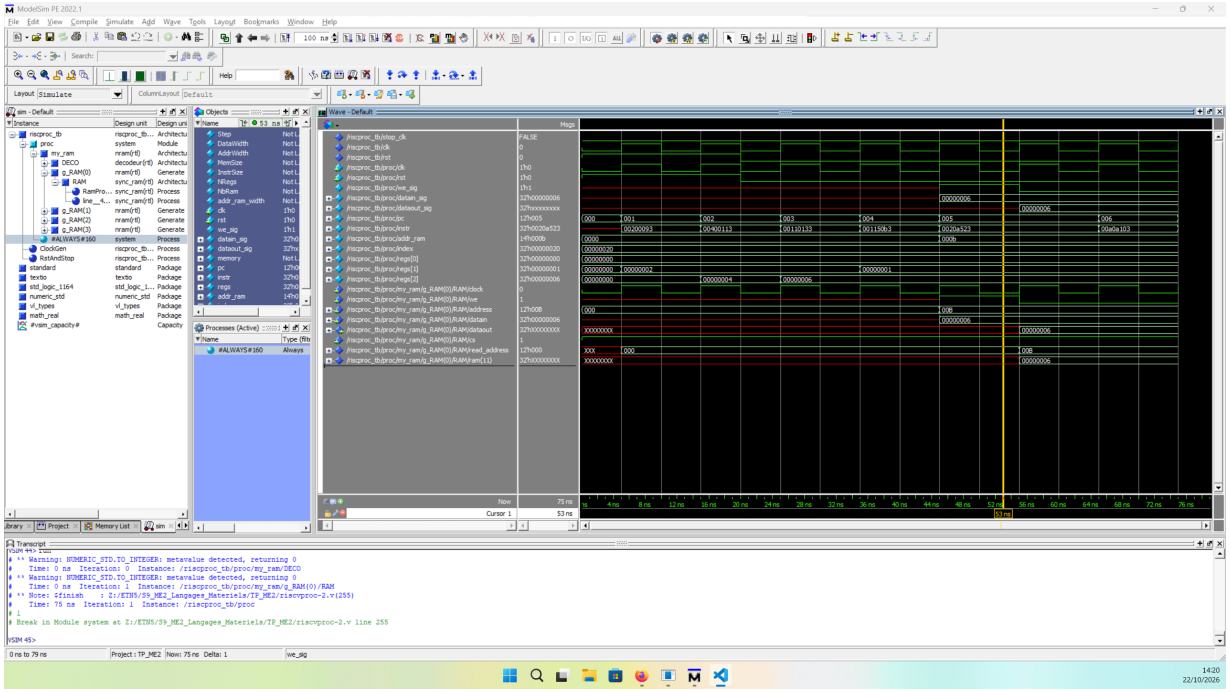


FIGURE 3.1 – Écriture et lecture dans le bloc mémoire `g_RAM(0)` à l'adresse 11 (`0x00b`).

Validation avec le deuxième programme : adressage multi-blocs dans `nram`.

Dans le premier programme, l'adresse accédée (11 = `14'b00_0000_0000_1011`) possède ses deux bits de poids fort (`addr_ram[13:12]`) à `00`, ce qui ne sollicite que le premier bloc mémoire `g_RAM(0)`.

Pour valider le décodage d'adresse et l'écriture dans un autre bloc de la `nram` (ici le bloc `g_RAM(2)`, sélectionné lorsque `addr_ram[13:12] = 2'b10`), une séquence d'instructions est ajoutée à la fin de `instr.mem` (Figure 3.2) :

- `0x00120213` (`addi x4, x4, 1`) : $x4 = 1$.
- `0x00d28293` (`addi x5, x5, 13`) : $x5 = 13$.
- `0x00521233` (`sll x4, x4, x5`) : $x4 = 1 \ll 13 = 8192$ (`14'b10_0000_0000_0000`), positionnant les bits `[13:12]` à `10` (sélection de la RAM 2) et les 12 bits de poids faible à 0.
- `0x00122023` (`sw x1, 0(x4)`) : écriture de $x1 = -2147483648$ à l'adresse 0 du bloc `g_RAM(2)`.

Le chronogramme de la Figure 3.3 nous confirme que la case `g_RAM(0)/RAM/ram(11)` conserve la valeur 6, tandis qu'à $t = 35$ ns, l'instruction `sw x1, 0(x4)` écrit la valeur `-2147483648` dans la case `g_RAM(2)/RAM/ram(0)`.

instr.mem | nram_tb.vhd | nram.vhd | ramchip.vhd | decodeur.vhd

instr.mem

```
1   // instructions at address 0x0
2   // converter https://luplab.gitlab.io/rvcodecjs/#q=sw+x2,+10(x1)&abi=false&isa=A
3   @0
4   00200093 // addi x1, x0, 2
5   00400113 // addi x2, x0, 4
6   00110133 // add x2, x2, x1
7   001150b3 // srl x1, x2, x1 // x1 = 1
8   0020a523 // sw x2, 10(x1)
9   00a0a103 // lw x2, 10(x1)
10
11  // division et reminder par 0
12  0230e033 // rem x0, x1, x3  // x0 = x1 = 1 (rem par 0)
13  0230c033 // div x0, x1, x3  // x0 = -1 (div par 0)
14
15  // division et reminder de -2**(L-1) par -1
16  01910113 // addi x2, x2, 25 // x2 = 31
17  002090b3 // sll x1, x1, x2 // x1 = -2**(L-1)
18  0200e133 // rem x2, x1, x0 // x2 = 0 (rem overflow)
19  0200c133 // div x2, x1, x0 // x2 = -2**(L-1) (div overflow)
20
21
22  // stockage en ram 2 et non pas 0
23  00120213    // addi x4, x4, 1   // x4 = 1
24  00d28293    // addi x5, x5, 13  // x5 = 13
25  00521233    // sll x4, x4, x5   // x4 = 10'0000'0000'0000 (selection ram 2)
26  00122023    // sw x1, 0(x4)
27
28  00100073 // ebreak
```

FIGURE 3.2 – Programme de test modifié (`instr.mem`) ciblant le bloc mémoire `g_RAM(2)`.

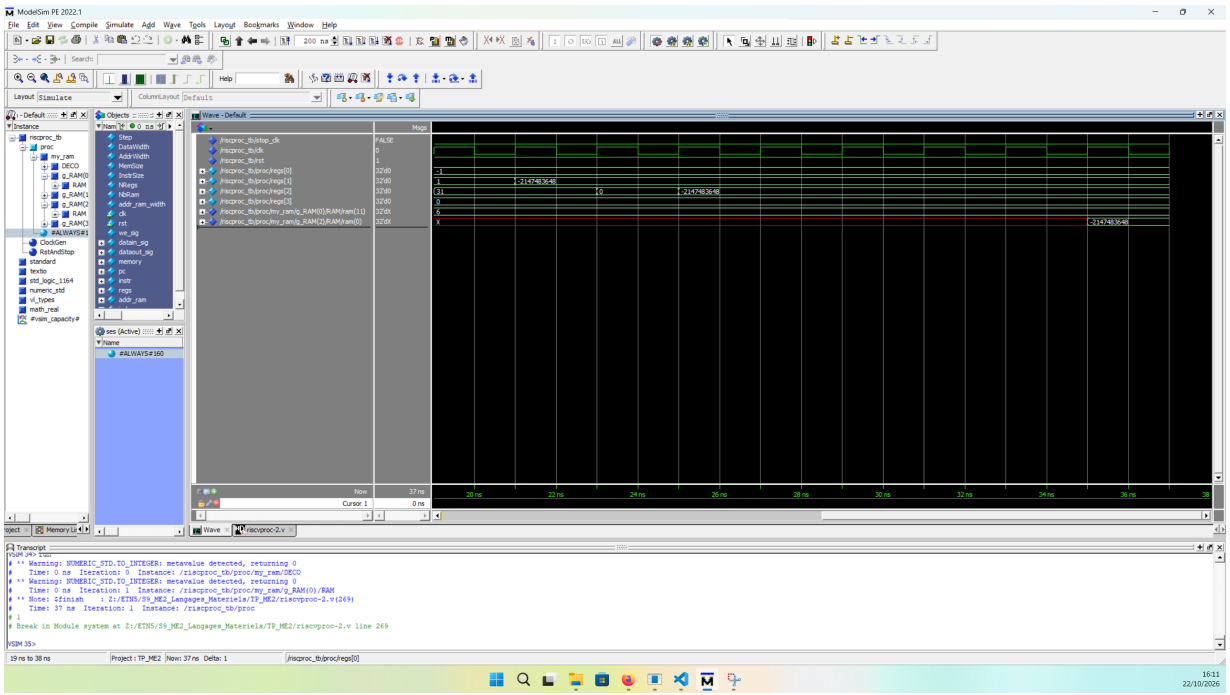


FIGURE 3.3 – Validation de l'écriture dans la case 0 du bloc `g_RAM(2)` à $t = 35$ ns.

4

Question pour un champion en matériel

Quelles sont les étapes nécessaires afin de rendre le modèle du système complet synthétisable sur une cible FPGA Xilinx ?

Pour rendre le système synthétisable sur un FPGA AMD (anciennement Xilinx) sous Vivado, les modifications suivantes sont nécessaires :

- **Ajout d'entrées/sorties physiques :** Le module `system` ne possède actuellement que `clk` et `rst`. Sans port de sortie externe, l'outil de synthèse supprimera la logique interne par optimisation. Il faut exposer des signaux vers les broches du FPGA (ex. : bus de données, LEDs, UART) et déclarer le brochage associé dans un fichier de contraintes (`.xdc`).
- **Suppression du code Verilog non synthétisable :**
  - Remplacer le contrôle d'événement `@(posedge clk)` imbriqué à l'intérieur du `case` de l'instruction `SW` par une machine à états finis.
  - Remplacer l'initialisation des registres et de `pc` dans le bloc `initial` par une initialisation synchrone ou asynchrone sur le signal `rst`.
  - Supprimer les commandes propres au simulateur (`$display`, `$finish`) et bloquer `pc` sur lui-même lors d'un `EBREAK` pour arrêter le programme.